Aprendizaje reforzado: SARSA, Q-learning y Deep Q-learning
El Paso del Contrabandista y el alunizaje con DQN

Carlos Ortega

27 de octubre de 2026

Universidad Iberoamericana
Ingeniería en Mecatrónica y Sistemas Ciberfísicos
Control Inteligente
Otoño 2026

Introducción

El objetivo de esta práctica es analizar y comparar el **comportamiento de riesgo** de una política *on-policy* (SARSA) y una *off-policy* (Q-learning) en una tarea de navegación táctica con incertidumbre, y después escalar a un espacio de estados continuo con **Deep Q-Learning** (DQN).

El primer ejercicio simula un robot móvil con ocho grados de libertad de movimiento en una malla de 20×20 con obstáculos y piso resbaloso. Se diseñó el mapa *El Paso del Contrabandista*: una barrera vertical separa el inicio A de la meta B y solo tiene una **brecha de una celda**, que se cruza en diagonal (camino corto y peligroso), mientras que un espacio abierto permite rodear la barrera (camino largo y seguro). La pregunta central es cuál de las dos rutas aprende cada algoritmo y por qué. El tema visual del sitio es americanista: el robot *busca el gol*, cada episodio es un *partido* y cada choque es una *tarjeta roja*.

El segundo ejercicio entrena un agente DQN en el entorno `LunarLander` de Gymnasium, cuyo estado $s \in \mathbb{R}^8$ es continuo, para aterrizar un módulo entre dos banderas. Se estudian sus fases de aprendizaje, el efecto de la penalización por combustible y la escalabilidad de una tabla Q frente a una red neuronal.

El procedimiento seguido fue: modelar ambos problemas como MDP, implementar los agentes en Python (NumPy y PyTorch), entrenarlos localmente en CPU con varias semillas, exportar los resultados y publicarlos en un sitio web interactivo con despliegue automático en GitHub Pages. El sitio incluye dos juegos 3D: uno donde SARSA y Q-learning juegan el mismo partido con sus tablas Q reales, y otro donde la red DQN entrenada en Python pilota el módulo lunar directamente en el navegador.

Sitio: `https://sebastiux.github.io/practiva7inteligente/`
Repositorio: `https://github.com/sebastiux/practiva7inteligente`

1. Especificaciones de diseño

Parámetro (Ejercicio 1)	En la práctica	Valor usado
Estados $\mathcal{S}$	malla $N \times N$ (sugerido 20×20)	$20 \times 20 = 400$ celdas
Acciones $\mathcal{A}$	$\{N, S, E, O, NE, NO, SE, SO\}$	8 acciones
Resbalamiento p_{slip}	0,1	0,1
Recompensa por llegar a la meta	$+100$	$+100$ (termina)
Colisión con obstáculo o pared	-100 (termina)	-100 (termina)
Paso cardinal / diagonal	$-1{,}0$ / $-1{,}41$	$-1{,}0$ / $-1{,}41$
Tasa de aprendizaje α	0,1	0,1
Factor de descuento γ	0,9	0,9
Exploración ε	0,1 (constante)	0,1 (constante)
Episodios	5,000–10,000	10,000
Semillas	no especificado	5 (de la 0 a la 4)
Límite de pasos por episodio	no especificado	400 (truncamiento)

Tabla 1: Parámetros del Ejercicio 1 (SARSA y Q-learning).

Parámetro (Ejercicio 2)	En la práctica	Valor usado
Entorno	`LunarLander-v2`	`LunarLander-v3`
Red neuronal	2 ocultas de 64 o 128	$8 \to 128 \to 128 \to 4$
Activación de las capas ocultas	ReLU	ReLU
Tasa de aprendizaje α (Adam)	0,0005	0,0005
Factor de descuento γ	0,99	0,99
Exploración ε	$1{,}0 \to 0{,}01$	$\times 0{,}995$ por episodio, mín. 0,01
Episodios	600–1,000	máx. 1,000 (paro: media ≥ 200)
Experience Replay	sí	buffer de 100,000, lote de 64
Red objetivo	no especificada	actualización suave, $\tau = 10^{-3}$
Frecuencia de aprendizaje	no especificada	cada 4 pasos
Evaluación	política greedy ($\varepsilon = 0$)	20 episodios

Tabla 2: Parámetros del Ejercicio 2 (DQN). `LunarLander-v2` ya no existe en Gymnasium 1.4; la versión 3 conserva el mismo estado, las mismas acciones y la misma recompensa, con correcciones de errores.

2. Marco teórico

2.1. Procesos de decisión de Markov

Un MDP es la tupla $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$: en cada paso t el agente observa S_t, elige A_t, recibe R_{t+1} y pasa a S_{t+1} con probabilidad $P(s' \mid s, a)$. Se cumple la *propiedad de Markov*: el futuro depende solo del estado actual. El objetivo es maximizar el **retorno descontado**

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, \qquad 0 \le \gamma < 1, \tag{1}$$

y la función de acción-valor $Q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$ mide qué tan buena es la acción a en el estado s si después se sigue π.

2.2. Aprendizaje por diferencias temporales

Los métodos de diferencias temporales (TD) aprenden Q sin conocer P, corrigiendo la estimación paso a paso hacia un *blanco* que combina la recompensa inmediata con el valor estimado del estado siguiente. La exploración se resuelve con una política ε-greedy: con probabilidad $1-\varepsilon$ se elige $\arg\max_a Q(s, a)$ y con probabilidad ε una acción al azar.

- **Q-learning (off-policy)** aprende el valor de la política óptima independientemente de las acciones exploratorias que tome:

$$Q(s, a) \leftarrow Q(s, a) + \alpha\Big[r + \gamma \max_{a'} Q(s', a') - Q(s, a)\Big]. \tag{2}$$

- **SARSA (on-policy)** aprende el valor de la política que realmente ejecuta, exploración incluida, porque usa la acción a' que la política ε-greedy sí elegirá en s':

$$Q(s, a) \leftarrow Q(s, a) + \alpha\Big[r + \gamma\, Q(s', a') - Q(s, a)\Big]. \tag{3}$$

La única diferencia entre (2) y (3) es el término del estado siguiente: $\max_{a'} Q(s', a')$ contra $Q(s', a')$. Esa diferencia determina la actitud de cada algoritmo ante el riesgo: Q-learning valora cada estado suponiendo que en el futuro siempre jugará la mejor acción, mientras que SARSA incorpora la probabilidad real de equivocarse al explorar.

2.3. Deep Q-learning

Cuando el estado es continuo, la tabla Q es inviable. DQN la reemplaza por una red neuronal con pesos w,

$$Q(s, a) \approx Q(s, a; w), \tag{4}$$

entrenada por retropropagación para minimizar el error cuadrático contra el blanco de Bellman:

$$\mathcal{L}(w) = \mathbb{E}\Big[\big(r + \gamma \max_{a'} Q(s', a'; w^-) - Q(s, a; w)\big)^2\Big]. \tag{5}$$

Se usan dos mejoras estándar sobre la formulación básica:

- **Experience Replay:** las transiciones $(s, a, r, s', \text{terminal})$ se guardan en un buffer circular y se entrena con mini-lotes aleatorios, lo que rompe la correlación temporal entre muestras consecutivas y reutiliza la experiencia.

- **Red objetivo:** la ecuación (4) de la práctica usa los mismos pesos w en el blanco y en la predicción, por lo que el blanco se mueve con cada paso de gradiente. Aquí el blanco se calcula con una copia lenta w^- actualizada suavemente, $w^- \leftarrow \tau w + (1-\tau)w^-$ (Mnih et al., 2015). No es parte del enunciado, pero estabiliza mucho el entrenamiento.

3. Modelado del problema

3.1. MDP del Grid World 8-DOF

- **Estados:** las 400 celdas $s = (\text{fila}, \text{columna})$. La barrera ocupa la columna 10 de la fila 0 a la 11, con la brecha en $(6, 10)$. El inicio es $A = (4, 6)$ y la meta $B = (7, 15)$; las filas 12 a 19 forman el espacio abierto para rodear (Figura 2).
- **Acciones:** 4 cardinales y 4 diagonales.
- **Transición con resbalamiento:** con probabilidad 0,9 el robot va a donde quiso; con probabilidad 0,1 patina y cae, con la misma probabilidad, en la dirección girada $-45°$, en la girada $+45°$ (celdas adyacentes a la deseada) o se queda en su lugar:

$$P(s' \mid s, a) = \begin{cases} 0{,}9 & s' = s + \Delta(a) \\ 0{,}1/3 & s' \in \{\, s + \Delta(a_{-45°}),\ s + \Delta(a_{+45°}),\ s \,\} \end{cases} \tag{6}$$

- **Recompensa:** $+100$ al llegar a B, -100 al chocar (el robot no avanza y el episodio termina), -1 por paso cardinal y $-1{,}41$ por paso diagonal. El costo se cobra según la acción *comandada*, aunque el robot patine.
- **Regla de colisión diagonal (decisión de diseño):** un movimiento choca solo si la celda *destino* es un obstáculo o queda fuera de la malla. Una diagonal puede pasar rozando las esquinas de dos obstáculos. Por eso la brecha se cruza en diagonal, $(5, 9) \xrightarrow{SE} (6, 10) \xrightarrow{SE} (7, 11)$, y por eso mismo es peligrosa: dentro de ella, una acción exploratoria hacia el norte o el sur termina en la barrera.

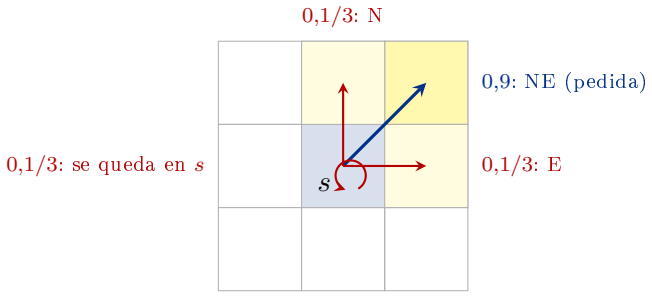


Figura 1: Modelo de resbalamiento para la acción NE: 0,9 a la celda pedida y 0,1/3 a cada celda adyacente a la deseada (N, E) o permanecer en s.

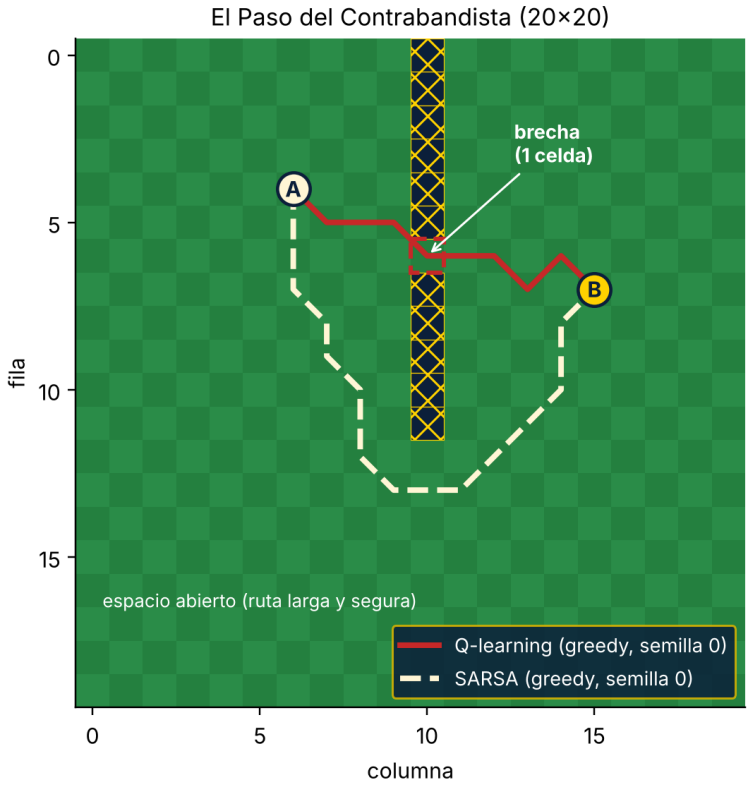


Figura 2: Mapa *El Paso del Contrabandista*. En rojo, la ruta greedy aprendida por Q-learning (cruza la brecha); en crema punteado, la de SARSA (rodea la barrera). Ambas siguen la política final de la semilla 0 sin resbalones.

La geometría se eligió empíricamente: se probaron varias versiones del mapa entrenando ambos agentes con las 5 semillas y se conservó la que separa las rutas de forma consistente. Como verificación, se resolvió el MDP exacto (con resbalamiento) por iteración de valor. La política greedy óptima cruza la brecha, con $V^*(A) = 29{,}96$ frente a 12,05 si la brecha se cierra, lo que coincide con lo aprendido por Q-learning ($\max_a Q(A, a) = 28{,}2$ en la semilla 0). Sin embargo, con $\gamma = 0{,}9$ el camino corto vale tanto que incluso la mejor política ε-greedy con $\varepsilon = 0{,}1$ cruzaría la brecha (18,4 frente a 5,97 por el rodeo). Es decir, el rodeo de SARSA no es su punto fijo asintótico, sino el resultado de su aprendizaje on-policy en 10,000 episodios; esto se discute en la Sección 6.

3.2. Ejemplo numérico de una actualización

Con la tabla Q final de Q-learning (semilla 0), el robot está en $s = (5, 9)$, justo antes de la brecha, ejecuta $a = SE$, no resbala, llega a $s' = (6, 10)$ y paga $r = -1{,}41$. El valor actual es $Q(s, a) = 51{,}54$.

$$\begin{aligned}
\textbf{Q-learning:}\quad & \max_{a'} Q(s', a') = Q(s', E) = 59{,}74, \\
& \text{blanco} = -1{,}41 + 0{,}9(59{,}74) = 52{,}36, \\
& Q(s, a) \leftarrow 51{,}54 + 0{,}1(52{,}36 - 51{,}54) = \mathbf{51{,}62}. \\
\textbf{SARSA:}\quad & a' = N \text{ (acción exploratoria)}, \quad Q(s', N) = -84{,}59, \\
& \text{blanco} = -1{,}41 + 0{,}9(-84{,}59) = -77{,}54, \\
& Q(s, a) \leftarrow 51{,}54 + 0{,}1(-77{,}54 - 51{,}54) = \mathbf{38{,}63}.
\end{aligned}$$

Con la misma experiencia, Q-learning casi no cambia su valoración de entrar a la brecha, mientras que SARSA la castiga, porque con $\varepsilon = 0{,}1$ de vez en cuando sí jugará esa acción mala. Repetida miles de veces, esta diferencia es la que separa las dos rutas.

3.3. MDP de LunarLander

- **Estado** $s \in \mathbb{R}^8$: posición (x, y), velocidades (v_x, v_y), ángulo y velocidad angular $(\theta, \dot{\theta})$ y contacto binario de cada pata $(l_{\text{izq}}, l_{\text{der}})$.
- **Acciones:** {0: nada, 1: propulsor izquierdo, 2: motor principal, 3: propulsor derecho}.
- **Recompensa densa:** premia acercarse a la plataforma despacio y nivelado, +10 por cada pata en tierra, +100 por aterrizar, −100 por estrellarse, y penaliza el combustible con −0,3 por cuadro de motor principal y −0,03 por cuadro de propulsor lateral.

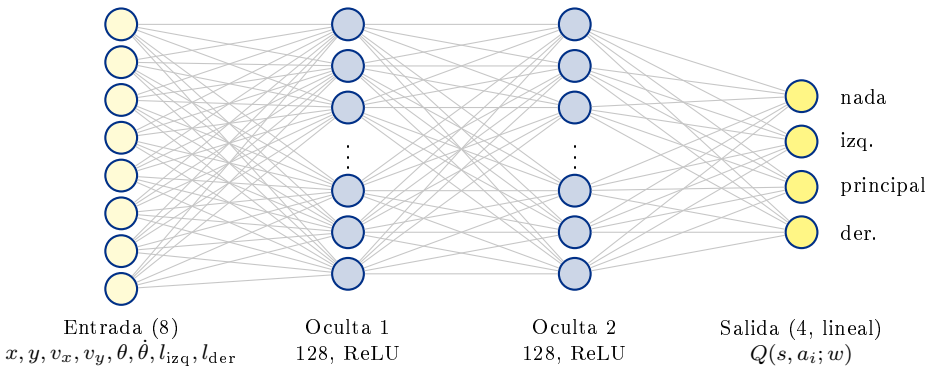


Figura 3: Arquitectura de la red: $8 \to 128 \to 128 \to 4$, con $(8 \cdot 128 + 128) + (128 \cdot 128 + 128) + (128 \cdot 4 + 4) = 18{,}180$ parámetros.

4. Implementación

La práctica se organizó en un repositorio con dos partes:

- `python/gridworld.py`: entorno 20×20 con 8 acciones, resbalamiento y la regla de colisión diagonal.
- `python/agents.py`: clase `AgenteRL`, configurable como `"sarsa"` o `"qlearning"`.
- `python/train_tabular.py`: entrena ambos agentes con 5 semillas y exporta curvas, matrices de visitas, tabla Q y política a JSON.
- `python/dqn_lunar.py`: DQN con PyTorch y Gymnasium; exporta la historia de entrenamiento, el modelo `.pt` y un video del aterrizaje.
- `python/export_policy.py`: exporta los pesos de la red a JSON y mide líneas base en Gymnasium.
- `web/`: sitio en Vite, React y TypeScript con gráficas de Plotly, ecuaciones en KaTeX y los juegos 3D (three.js). Incluye el port a TypeScript del Grid World y de la física de LunarLander.
- `.github/workflows/deploy.yml`: pruebas (pytest y Vitest), compilación y despliegue automático en GitHub Pages con cada *push*.

```python
def transition(self, pos, real_action, commanded):
    """Devuelve (nueva_pos, recompensa, terminal)."""
    if real_action == STAY:                          # resbalo y se quedo
        return pos, step_cost(commanded), False
    dr, dc = ACTION_DELTAS[real_action]
    nr, nc = pos[0] + dr, pos[1] + dc
    if self.blocked(nr, nc):                         # solo se revisa el destino
        return pos, R_CRASH, True                    # -100 y termina
    if (nr, nc) == self.goal:
        return (nr, nc), R_GOAL, True                # +100 y termina
    return (nr, nc), step_cost(commanded), False     # -1 o -1.41

def sample_real_action(self, action):
    if self.rng.random() >= self.p_slip:
        return action
    k = self.rng.integers(3)                         # -45, +45 o quedarse
    return (rotate(action, -1), rotate(action, 1), STAY)[k]
```

Listing 1: Transición del Grid World con la regla de colisión (extracto de `gridworld.py`).

```python
def target(self, r, s_next, a_next, done):
    if done:
        return r
    if self.algo == "qlearning":
        # Ec. (1): max_a' Q(s', a'), independiente de la accion que se tomara
        return r + self.gamma * self.Q[s_next].max()
    # Ec. (2): Q(s', a') con la a' que la politica epsilon-greedy SI eligio
    return r + self.gamma * self.Q[s_next, a_next]

def update(self, s, a, r, s_next, a_next, done):
    td = self.target(r, s_next, a_next, done) - self.Q[s, a]
    self.Q[s, a] += self.alpha * td
    return td
```

Listing 2: Blanco TD y actualización del `AgenteRL` (extracto de `agents.py`).

```python
def learn(self):
    s, a, r, s2, d = self.memory.sample(self.batch, self.rng)    # Experience
↪ Replay
    with torch.no_grad():
        # Blanco de Bellman de la ec. (4), evaluado con la red objetivo w-
        target = r + self.gamma * self.q_target(s2).max(1).values * (1 - d)
    q_sa = self.q(s).gather(1, a.unsqueeze(1)).squeeze(1)
    loss = F.mse_loss(q_sa, target)                # E[(blanco - Q(s,a;w))^2]
    self.opt.zero_grad()
    loss.backward()
    nn.utils.clip_grad_norm_(self.q.parameters(), 10.0)
    self.opt.step()
    with torch.no_grad():                          # w- <- tau*w + (1-tau)*w-
        for p_t, p in zip(self.q_target.parameters(), self.q.parameters()):
            p_t.mul_(1 - self.tau).add_(self.tau * p)
    return float(loss)
```

Listing 3: Paso de aprendizaje de DQN con red objetivo (extracto de `dqn_lunar.py`).

Para la evaluación greedy y el mapa de calor, el truncamiento por tiempo (400 pasos en el Grid World, 1,000 en LunarLander) no se trata como estado terminal, de modo que no se "corta" el valor futuro de forma artificial.

4.1. Juegos 3D

El sitio incluye dos juegos que usan los resultados reales de la práctica:

- **El Paso del Contrabandista 3D.** La malla es una cancha, la barrera es una fila de defensas y la brecha está marcada en rojo. En el modo *Duelo*, SARSA y Q-learning juegan a la vez con sus tablas Q finales de Python, con ε y resbalón activos. El robot patina al resbalar, recibe tarjeta roja al chocar y celebra el gol al llegar a B. Además hay un modo para entrenar un agente en vivo, con mapa de calor sobre el pasto y editor de cancha, y un modo en el que el usuario controla al robot.
- **Alunizaje DQN.** Se portó la física de `LunarLander` a TypeScript con `planck.js`, una versión del motor Box2D que usa Gymnasium, con las mismas constantes, terreno aleatorio, recompensa y observación normalizada. La red de PyTorch se exportó a JSON y se evalúa en el navegador a 50 Hz; las barras del HUD muestran sus cuatro $Q(s, a)$ en cada instante. También se puede pilotar manualmente.

```
qValues(obs) {
  let x = Float32Array.from(obs)
  this.layers.forEach((L, li) => {
    const y = new Float32Array(L.nOut)
    for (let o = 0; o < L.nOut; o++) {
      let s = L.b[o]
      const row = o * L.nIn
      for (let i = 0; i < L.nIn; i++) s += L.W[row + i] * x[i]
      y[o] = li < this.layers.length - 1 ? Math.max(0, s) : s    // ReLU
    }
    x = y
  })
  return Array.from(x)           // Q(s, a) para las 4 acciones
}
```

Listing 4: Evaluación de la red DQN en el navegador (extracto de `policy.ts`).



Figura 4: Juego *El Paso del Contrabandista 3D* en modo Duelo: Q-learning (amarillo) se dirige a la brecha mientras SARSA (azul) baja para rodear la barrera. El marcador muestra goles y tarjetas rojas de cada algoritmo.

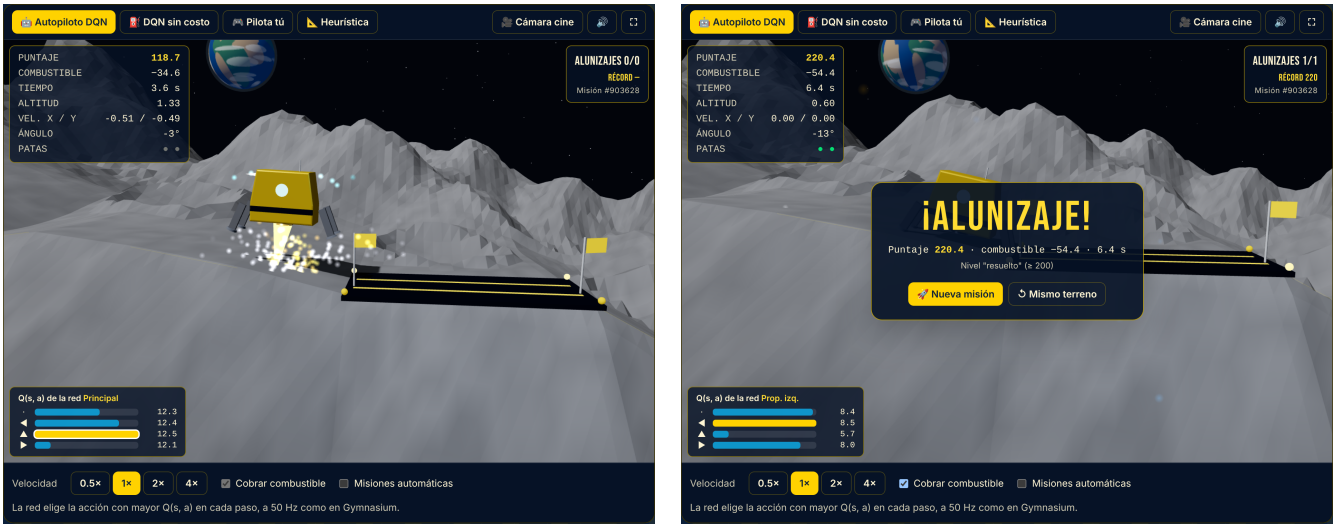


Figura 5: Juego *Alunizaje DQN*: la red entrenada en Python pilota el módulo en el navegador. Izquierda: motor principal encendido sobre la plataforma, con los $Q(s, a)$ en vivo. Derecha: aterrizaje con 220,4 puntos.

5. Resultados

5.1. Entregable 1: comparativa de convergencia

Ambos agentes se entrenaron en el mismo mapa con $\alpha = 0{,}1$, $\gamma = 0{,}9$, $\varepsilon = 0{,}1$ constante y 10,000 episodios, repitiendo con 5 semillas. Cada semilla es la misma para los dos algoritmos (mismo generador del entorno y del agente). La Figura 6 muestra la media de las 5 semillas con una banda de ± 1 desviación estándar.

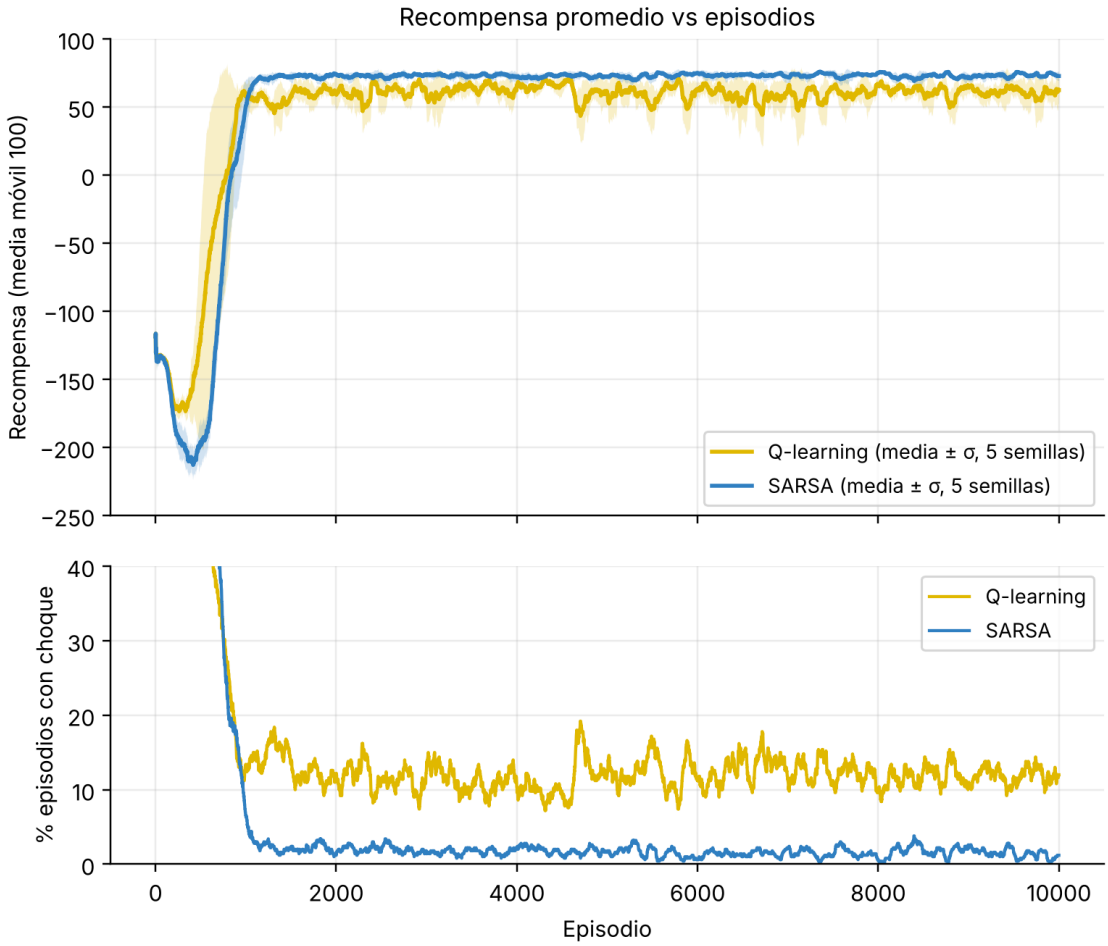


Figura 6: Arriba: recompensa promedio (media móvil de 100 episodios) contra episodios. Abajo: porcentaje de episodios que terminan en choque.

Algoritmo	Sem. 0	Sem. 1	Sem. 2	Sem. 3	Sem. 4	Media $\pm$ σ	Choques
Q-learning	64,7	53,2	66,1	52,6	75,1	$62{,}3 \pm 8{,}5$	10,2 %
SARSA	76,1	71,8	73,0	75,6	66,9	$\mathbf{72{,}7 \pm 3{,}3}$	**1,1 %**

Tabla 3: Recompensa promedio de los últimos 100 episodios de entrenamiento por semilla, y porcentaje de choques en la evaluación con $\varepsilon = 0{,}1$.

¿Cuál converge a un valor más alto durante el entrenamiento y por qué? SARSA, con 72,7 puntos por episodio contra 62,3 de Q-learning. La razón está en la diferencia entre (2) y (3):

- SARSA valora cada jugada sabiendo que el 10 % de las veces tirará una acción al azar, y aprende una ruta que tolera esos errores.
- Q-learning aprende el valor de la política greedy óptima e ignora que él mismo sigue explorando. Elige la ruta corta por la brecha, donde cada acción aleatoria o resbalón termina en un choque de -100 sin el $+100$ de la meta.

Por eso la curva de Q-learning queda más abajo, es más ruidosa ($\sigma = 8{,}5$ contra 3,3) y acumula unas 9 veces más choques.

Si se apaga la exploración el resultado se invierte: con $\varepsilon = 0$, la política de Q-learning promedia 81,1 contra 78,9 de SARSA, porque Q-learning sí aprendió el camino más corto. Los primeros $\sim$ 1,500 episodios son muy negativos porque, con $Q = 0$ inicial, el agente deambula hasta el límite de 400 pasos antes de descubrir la meta.

5.2. Entregable 2: mapas de calor de trayectorias

La matriz de visitas cuenta cuántas veces el robot pisó cada celda una vez entrenado. Se obtuvo en una evaluación de 1,000 episodios por semilla **con** $\varepsilon = 0{,}1$ **y resbalón activos**, sin actualizar Q, y se sumaron las 5 semillas.

¿Por qué evaluar con $\varepsilon = 0{,}1$ **y no con la política greedy?** Con $\varepsilon = 0$ cada agente repite casi siempre la misma trayectoria y el riesgo de la ruta elegida casi no aparece en el mapa: no se ven los choques ni la nube de desviaciones alrededor de la brecha. Con $\varepsilon = 0{,}1$ se observa la distribución real de trayectorias, incluidos los errores, que es justo lo que cada algoritmo valoró distinto.

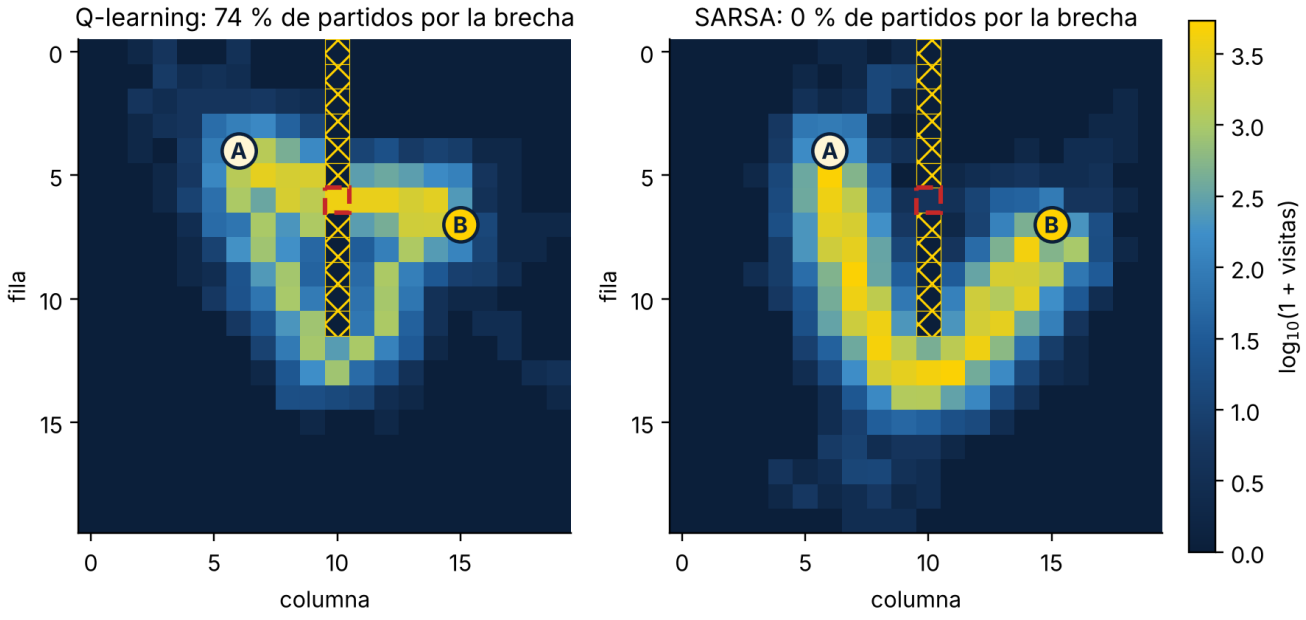


Figura 7: Mapas de calor de visitas (escala $\log_{10}$, suma de 5 semillas) en evaluación con $\varepsilon = 0{,}1$. La barrera está rayada y la brecha marcada en rojo.

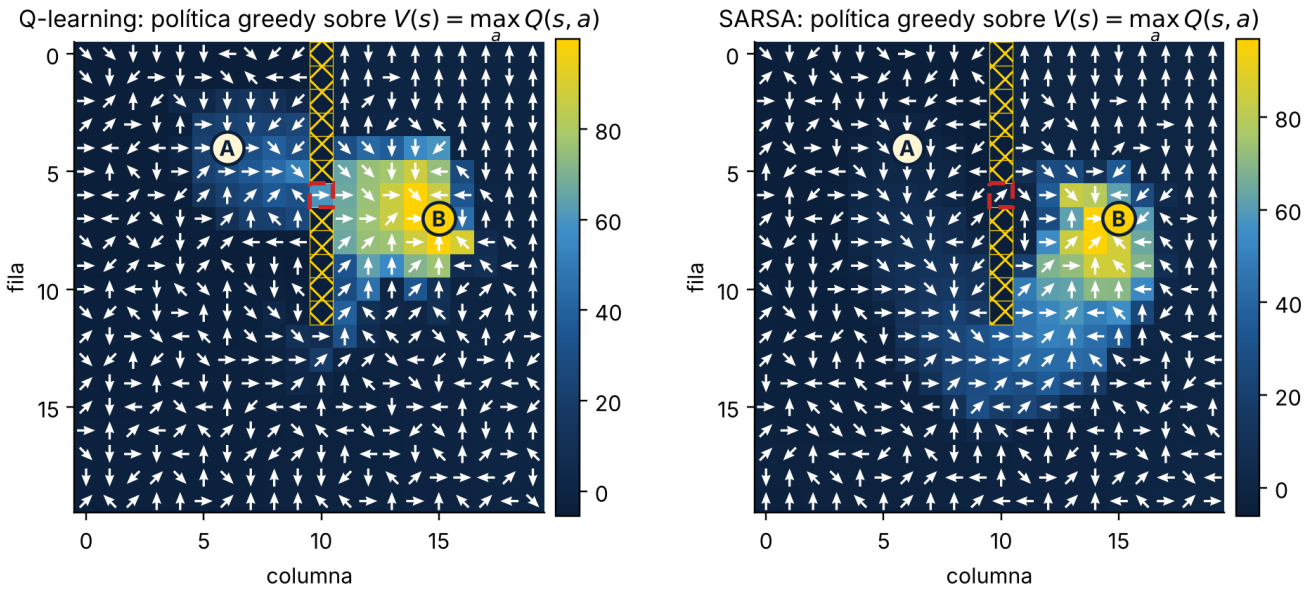


Figura 8: Política greedy aprendida (flechas) sobre $V(s) = \max_a Q(s,a)$, semilla 0. Q-learning dirige las flechas en diagonal hacia la brecha; SARSA las desvía hacia el espacio abierto.

Métrica	Q-learning, $\varepsilon = 0{,}1$	SARSA, $\varepsilon = 0{,}1$	Q-learning, $\varepsilon = 0$	SARSA, $\varepsilon = 0$
Recompensa media	65,7	**73,4**	**81,1**	78,9
Llega a la meta	89,8 %	98,9 %	96,7 %	100 %
Choques	10,2 %	1,1 %	3,3 %	0 %
Usa la brecha	74 %	0,1 %	77 %	0 %
Pasos por episodio	12,6	21,6	11,2	18,8

Tabla 4: Evaluación sin aprendizaje: 1,000 episodios por semilla, promedio de las 5 semillas.

Hipótesis verificada.

- **Q-learning corta esquinas:** el 74 % de sus episodios pasa por la brecha de una celda, rozando las esquinas de la barrera. En cuatro de las cinco semillas su política greedy usa la brecha; en la semilla 2 nunca la descubrió con ε constante y se quedó con la ruta larga.
- **SARSA mantiene un colchón de seguridad:** prácticamente nunca usa la brecha (0,1 %). Rodea la barrera por abajo, sin pegarse a su punta, y se mantiene lejos de las paredes exteriores. Su $V(s)$ en la brecha es mucho más bajo porque ahí la exploración se paga con tarjeta roja.

5.3. Entregable 3: discusión sobre robótica real

En un robot físico costoso que comparte espacio con personas se elegiría **SARSA**, o en general un método on-policy y conservador. En el mundo real el robot nunca deja de tener ruido: patina, los sensores fallan y, mientras aprende o se adapta, sigue explorando. Q-learning optimiza el valor de una política ideal que supone que nunca se equivoca, lo que produce rutas pegadas a las esquinas donde un solo error cuesta un choque; aquí ese choque vale -100, pero en la realidad es un robot roto o una persona lastimada. SARSA incorpora en $Q(s', a')$ la probabilidad real de equivocarse y por eso es **aversa al riesgo** (*risk averse*): sacrifica un poco de recompensa esperada (una ruta más larga) a cambio de reducir mucho la varianza y la probabilidad de un desenlace catastrófico. En nuestros datos eso se tradujo en unas 9 veces menos choques con $\varepsilon = 0{,}1$. En robótica de servicio el costo de un fallo es asimétrico e irreversible, así que ese colchón vale más que unos pasos ahorrados. Si se usara Q-learning, convendría apagar la exploración al desplegar y añadir restricciones de seguridad explícitas, como zonas prohibidas, márgenes o penalizaciones de proximidad.

5.4. Entregable DQN 1: curva de aprendizaje

La red se entrenó en CPU durante 128 s con la semilla 0 y se detuvo en el **episodio 526**, cuando la media móvil de 100 episodios superó +200 (con ε ya en 0,072). La Figura 9 muestra la recompensa por episodio, la media móvil y ε, con las fases del aprendizaje detectadas automáticamente en los datos (Tabla 5).

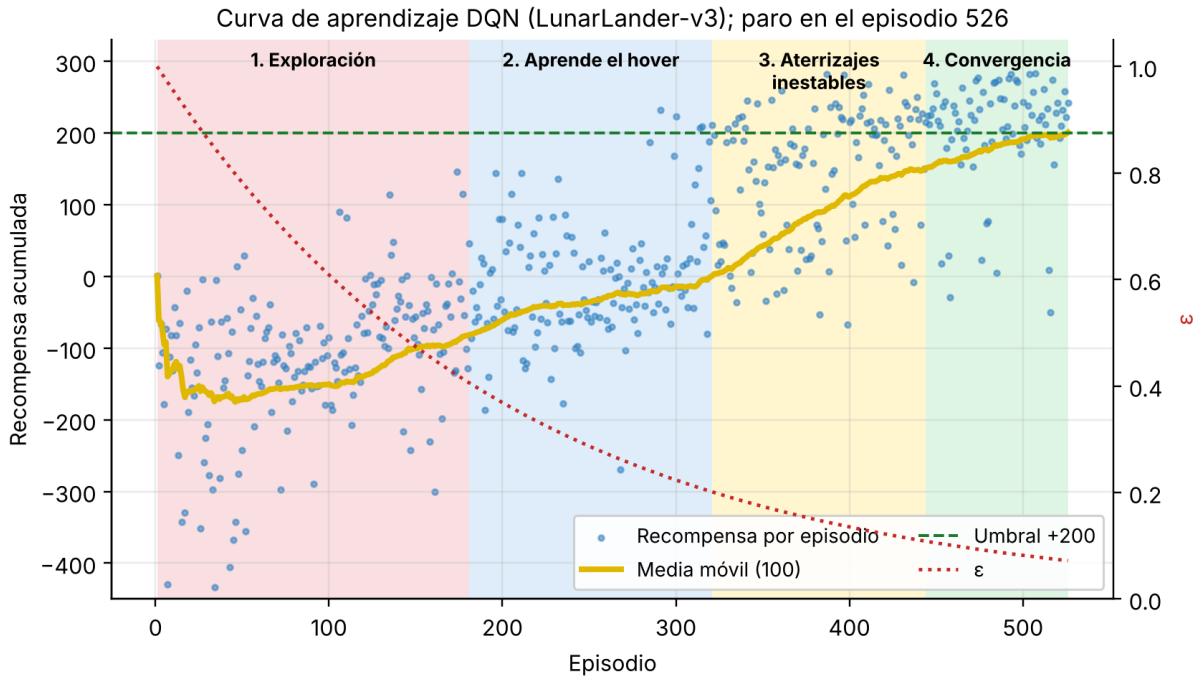


Figura 9: Curva de aprendizaje de DQN con las cuatro fases sombreadas.

Fase	Episodios	Comportamiento observado
1. Exploración	1–181	ε alto: acciones casi aleatorias; el módulo choca en ~ 100 pasos y la media ronda -150.
2. Aprende el *hover*	181–321	Descubre que flotar evita el -100: hasta el 86 % de los episodios llega al límite de 1,000 pasos. Inicia cuando $\geq 10\,\%$ de los últimos 50 episodios agotan el tiempo.
3. Aterrizajes inestables	321–444	Empieza a posarse (de 8 % a 80 % de aterrizajes), a veces rebota o se inclina. Inicia cuando $\geq 20\,\%$ de los últimos 50 episodios aterrizan.
4. Convergencia	444–526	Aterrizajes consistentes entre las banderas; la media móvil pasa de 150 a $+200$.

Tabla 5: Fases del aprendizaje del DQN.

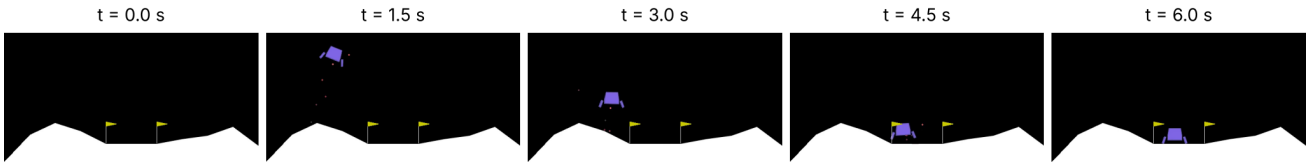


Figura 10: Aterrizaje con la política greedy final en Gymnasium (recompensa 294,3).

La evaluación greedy ($\varepsilon = 0$) en 20 episodios dio $\mathbf{160{,}9 \pm 79{,}6}$, con 55 % de aterrizajes. Queda por debajo de 200 aunque la media de entrenamiento lo superó: al parar justo en el umbral, el agente conserva algo del hábito de la fase 2, y 7 de los 20 episodios greedy se quedan flotando hasta los 1,000 pasos. Durante el entrenamiento el ε restante lo sacaba del *hover*; con $\varepsilon = 0$ no hay nada que lo empuje. Para comprobar que no es un efecto de la semilla se repitió el entrenamiento completo con cuatro semillas (Tabla 6). Con 100 episodios de evaluación, la red de la semilla 0 obtiene $193{,}3 \pm 59{,}5$ y 77 % de aterrizajes (Tabla 9).

Semilla	Paro (media ≥ 200)	Greedy, media $\pm\ \sigma$	Aterrizajes	Se queda flotando
0	episodio 526	$160{,}9 \pm 79{,}6$	55 %	7/20
1	episodio 500	$196{,}5 \pm 89{,}3$	90 %	1/20
2	episodio 570	$215{,}7 \pm 44{,}3$	90 %	2/20
3	episodio 478	$199{,}7 \pm 75{,}2$	85 %	2/20
Promedio	**episodio 519**	**193,2**	**80 %**	—

Tabla 6: Repetición del entrenamiento DQN con cuatro semillas (evaluación greedy de 20 episodios).

5.5. Entregable DQN 2: análisis de la función de recompensa

La recompensa resta 0,3 por cada cuadro con el motor principal y 0,03 por cada cuadro con un propulsor lateral. Un aterrizaje enciende el motor cientos de veces, de modo que este término suma decenas de puntos y **define el estilo de vuelo**:

- **Con penalización**, la política óptima es eficiente: deja caer el módulo casi en caída libre, frena con ráfagas cortas y tardías del motor principal, corrige el ángulo con los propulsores laterales (diez veces más baratos) y apaga todo en cuanto las patas tocan el suelo.
- **Con costo cero**, encender motores es gratis y emerge el *hover* prolongado: mantenerse suspendido es una forma segura de evitar el -100 y no cuesta nada. El agente usa el motor principal de más, desciende lento y tarda en asentarse; con $\gamma = 0{,}99$ hay poco apuro por cobrar el $+100$ pronto. En un sistema real eso significa combustible desperdiciado y menos autonomía.

Para comprobarlo se entrenó una variante con un *wrapper* que devuelve la penalización de combustible, con el mismo código, hiperparámetros y semilla.

Política greedy, 20 episodios	Con costo (original)	Costo = 0
Encendidos del motor principal por episodio	175	**350**
Encendidos de propulsores laterales	324	217
Duración media (pasos)	631	691
Episodios que agotan los 1,000 pasos	7/20	4/20
Aterrizajes	55 %	75 %
Recompensa que optimiza	$160{,}9 \pm 79{,}6$	$239{,}4 \pm 62{,}2$
Recompensa original (con combustible)	$160{,}9 \pm 79{,}6$	$128{,}1 \pm 80{,}5$
Episodio en que la media móvil llega a 200	526	404

Tabla 7: Efecto de la penalización por combustible.

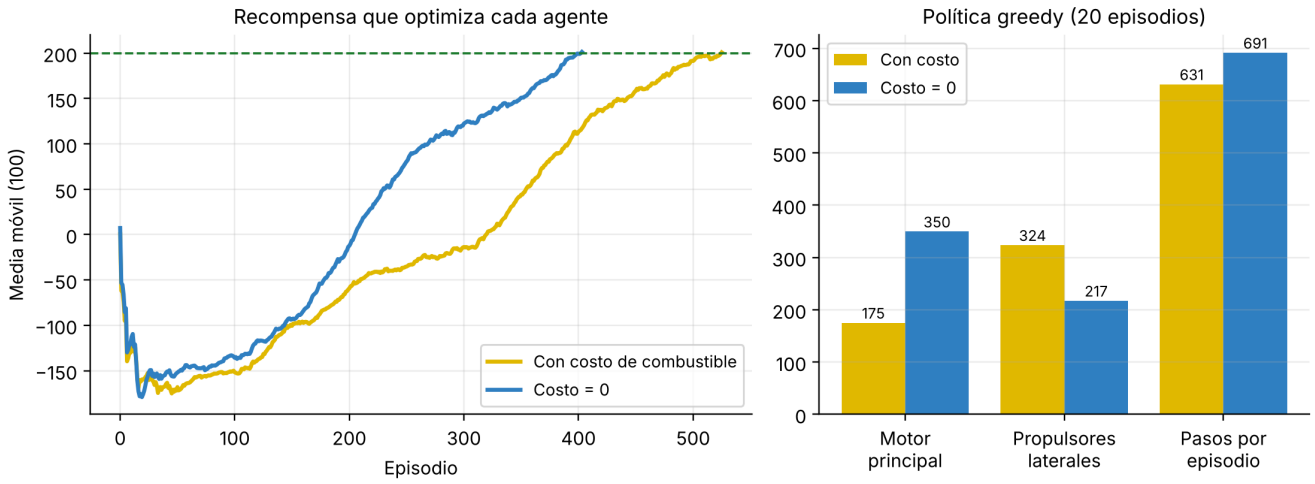


Figura 11: Izquierda: media móvil de la recompensa que optimiza cada agente. Derecha: uso de motores y duración de los episodios con la política greedy.

Sin costo, el agente enciende el motor principal el **doble** de veces y sus episodios duran un 10 % más. Su puntaje "inflado" de 239,4 se desploma a 128,1 cuando se le cobra la gasolina. Además alcanza el umbral antes (episodio 404 contra 526): el problema es más fácil porque explorar con los motores no se castiga, pero la política resultante es derrochadora. Como matiz, en esta corrida el agente *con* costo se queda flotando más veces (7/20 contra 4/20) porque se detuvo justo en el umbral; la diferencia de estilo se ve en cuánto empuja, no en si termina el episodio.

5.6. Entregable DQN 3: escalabilidad tabular frente a redes profundas

Si cada variable continua del módulo se discretiza en 20 segmentos:

Las 8 variables: $|\mathcal{S}| = 20^8 = 2{,}56 \times 10^{10}$ estados, $|\mathcal{S}| \cdot |\mathcal{A}| = 2{,}56 \times 10^{10} \times 4 = 1{,}024 \times 10^{11}$ entradas, $1{,}024 \times 10^{11} \times 4\,\mathrm{B} = 4{,}10 \times 10^{11}\,\mathrm{B} \approx 410$ GB.

Patas binarias: $|\mathcal{S}| = 20^6 \times 2^2 = 2{,}56 \times 10^8$ estados, $2{,}56 \times 10^8 \times 4 = 1{,}024 \times 10^9$ entradas, $1{,}024 \times 10^9 \times 4\,\mathrm{B} = 4{,}10 \times 10^9\,\mathrm{B} \approx 4{,}1$ GB.

Red neuronal: 18,180 parámetros $\times\, 4\,\mathrm{B} = 72{,}720\,\mathrm{B} \approx 72{,}7$ KB.

Representación	Entradas / parámetros	Memoria (float32)	Veces la red
Tabla Q, 8 variables × 20	$1{,}024 \times 10^{11}$	≈ 410 GB	$\approx 5{,}6 \times 10^6$
Tabla Q, patas binarias	$1{,}024 \times 10^9$	≈ 4,1 GB	$\approx 5{,}6 \times 10^4$
Red DQN 8-128-128-4	18,180	≈ 72,7 KB	1

Tabla 8: Memoria requerida por una tabla Q discretizada frente a la red neuronal.

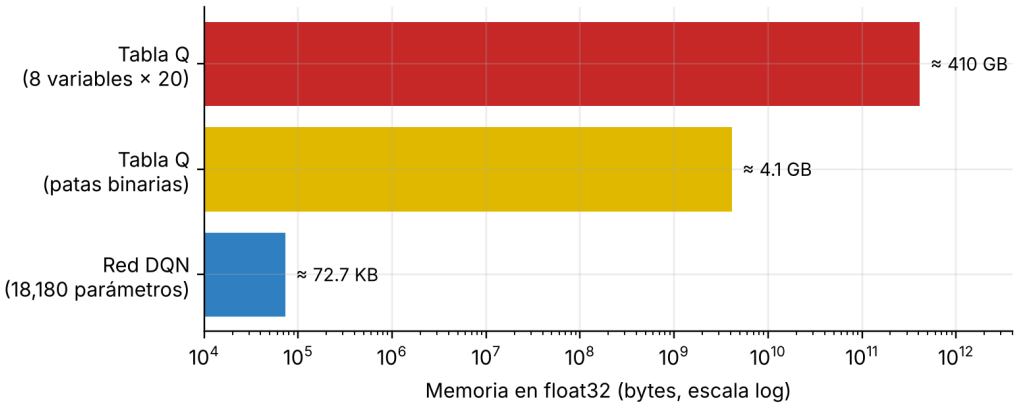


Figura 12: Comparación de memoria en escala logarítmica.

La memoria es solo la mitad del problema. Una tabla necesita *visitar* cada celda muchas veces para estimarla, y con 10^8 a 10^{10} estados casi todas quedarían sin datos; además, con 20 segmentos la física se vuelve "pixelada" (el ángulo cambiaría en saltos de $\sim 18°$). La red **generaliza**: estados cercanos comparten pesos, así que lo aprendido en uno se transfiere a sus vecinos. Por eso bastaron unos 500 episodios en lugar de miles de millones de visitas, lo que justifica el aproximador neuronal.

5.7. Validación del port de LunarLander

Para que el juego del sitio sea fiel al experimento, se comparó el port en `planck.js` contra Gymnasium con 100 episodios por lado. Como los generadores aleatorios son distintos, los terrenos no coinciden y se comparan distribuciones.

Controlador	Gymnasium (Python, Box2D)	Navegador (planck.js)
Heurística de Gymnasium	$252{,}8 \pm 73{,}1$, 95 % aterrizajes	$255{,}3 \pm 67{,}8$, 97 % aterrizajes
DQN (red principal)	$193{,}3 \pm 59{,}5$, 77 % aterrizajes	$196{,}4 \pm 63{,}6$, 76 % aterrizajes
DQN sin costo*	$229{,}8 \pm 80{,}5$, 70 % aterrizajes	$219{,}0 \pm 83{,}3$, 62 % aterrizajes

Tabla 9: Validación estadística del port. *Evaluada con la recompensa sin costo de combustible con la que se entrenó.

Las diferencias están dentro del ruido de 100 episodios, lo que confirma que la red entrenada en Python se comporta igual en el navegador.

6. Discusión

- **On-policy frente a off-policy es una actitud ante el riesgo.** Con el mismo mapa, los mismos hiperparámetros y las mismas semillas, la única diferencia en el código es una línea (el blanco TD), y basta para que los agentes elijan rutas opuestas. Q-learning aprende el valor de lo que sería óptimo *si no hubiera errores*; SARSA, el valor de lo que realmente hace, *errores incluidos*.

- **El ruido es lo que separa a los algoritmos.** Sin resbalón ni exploración, ambos convergerían a la ruta corta. El resbalón afecta a los dos por igual, porque es parte del entorno; la exploración solo la "ve" SARSA. Por eso el mapa de calor se midió con $\varepsilon = 0{,}1$: con la política greedy pura la diferencia de riesgo casi desaparece.

- **El rodeo de SARSA es un efecto del aprendizaje, no de su óptimo asintótico.** La iteración de valor muestra que, en este mapa, incluso la mejor política ε-greedy cruzaría la brecha (18,4 frente a 5,97). SARSA, en cambio, terminó con $\max_a Q(A, a) \approx 1{,}0$, muy lejos de ese punto fijo. La razón es que los choques causados por acciones exploratorias entran directamente en su blanco: en el ejemplo numérico, una sola actualización bajó Q de 51,54 a 38,63. La brecha empieza a verse mala, SARSA deja de visitarla y, con α y ε constantes, casi nunca vuelve a corregir esas estimaciones. Q-learning, con el máx, "perdona" las jugadas exploratorias y sí converge al óptimo greedy (28,2 frente a 29,96). La teoría indica que, con exploración suficiente y una tasa de aprendizaje decreciente, SARSA terminaría cruzando la brecha en este mapa; el efecto observado es transitorio, aunque persiste durante los 10,000 episodios. Para un robot real esta cautela aprendida sigue siendo deseable, porque no vuelve a intentar lo que le salió mal.

- **El diseño del mapa importa.** Las primeras versiones del mapa, con una brecha de tres celdas en diagonal, eran tan difíciles de descubrir con ε constante que Q-learning nunca encontraba el atajo. Incluso en el mapa final, una semilla de Q-learning nunca encontró la brecha, lo que muestra la limitación de una exploración ε-greedy sin decaimiento.

- **El criterio de paro de DQN es engañoso.** Detener el entrenamiento justo al cruzar la media móvil de 200 deja una política greedy que todavía conserva el hábito de flotar. La media de entrenamiento incluye el efecto de ε, que saca al módulo del *hover*. Una evaluación greedy con más episodios, o seguir entrenando con ε mínimo, da una imagen más fiel.

- **La recompensa moldea el comportamiento.** Quitar el costo de combustible no cambia el objetivo de aterrizar, pero duplica el uso del motor principal. El agente optimiza exactamente lo que se le pide, no lo que el diseñador "quiso decir".

7. Conclusiones

Los resultados del primer ejercicio coinciden con lo que anticipa la teoría. En *El Paso del Contrabandista*, SARSA terminó el entrenamiento con más recompensa que Q-learning ($72{,}7 \pm 3{,}3$ contra $62{,}3 \pm 8{,}5$ en los últimos 100 episodios) y, evaluado con $\varepsilon = 0{,}1$, chocó casi diez veces menos. Los mapas de calor muestran el porqué: Q-learning se mete por la brecha en tres de cada cuatro episodios, mientras que SARSA la evita y rodea la barrera por abajo. Al apagar la exploración la comparación se invierte y la política de Q-learning rinde más (81,1 contra 78,9), así que ninguno de los dos algoritmos es mejor en todos los sentidos: la elección depende de si el agente va a seguir equivocándose una vez desplegado.

El resultado menos esperado fue que el rodeo de SARSA no corresponde a su óptimo teórico. Al resolver el MDP por iteración de valor se encontró que, en este mapa, incluso la mejor política ε-greedy cruzaría la brecha. Lo observado se debe a la forma en que SARSA aprende: los choques de las primeras exploraciones hunden sus valores cerca de la barrera y, como después casi no vuelve a pasar por ahí, esas estimaciones no se corrigen. Para un robot costoso que opera entre personas esa cautela es justamente lo deseable, por lo que la recomendación sigue siendo SARSA, pero no debe confundirse con una garantía del algoritmo.

En LunarLander, la red de 18,180 parámetros alcanzó la media móvil de 200 en el episodio 526, con unos dos minutos de entrenamiento en CPU, y en la curva se distinguen las etapas de exploración, *hover*, aterrizajes inestables y convergencia. La evaluación greedy quedó por debajo del umbral ($160{,}9 \pm 79{,}6$ en 20 episodios) porque el módulo todavía se queda flotando en algunos intentos, lo que indica que detener el entrenamiento justo al cruzar 200 es un criterio optimista. Sin el costo de combustible el agente usó el doble de veces el motor principal, y la comparación de memoria, entre 4,1 y 410 GB para una tabla Q contra unos 73 KB para la red, muestra que en un espacio continuo no hay alternativa práctica a aproximar la función de valor.

Por último, portar la física al navegador y comprobar que la red obtiene en él los mismos resultados que en Gymnasium permitió convertir los experimentos en juegos interactivos, donde la diferencia de comportamiento entre los algoritmos se aprecia de forma directa.

Declaración sobre el uso de inteligencia artificial

Durante el desarrollo de esta práctica se utilizó Claude (Anthropic, 2026) como herramienta de apoyo en la programación de los agentes, del sitio web y de los juegos 3D, en el port de la física de LunarLander, en la configuración del despliegue en GitHub Pages, en la generación de las figuras y en la redacción y estructuración de este documento. La formulación del problema, la verificación de los resultados y la revisión del contenido fueron realizadas y validadas por los autores.

8. Referencias

- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2a ed.). MIT Press. Sección 6.5, ejemplo 6.6 *Cliff Walking*, p. 132.
- Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3–4), 279–292. `https://doi.org/10.1007/BF00992698`
- Rummery, G. A., & Niranjan, M. (1994). *On-line Q-learning using connectionist systems* (Reporte técnico CUED/F-INFENG/TR 166). Departamento de Ingeniería, Universidad de Cambridge.
- Lin, L.-J. (1992). Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8(3–4), 293–321. `https://doi.org/10.1007/BF00992699`
- Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533. `https://doi.org/10.1038/nature14236`
- Towers, M., Kwiatkowski, A., Terry, J., et al. (2024). *Gymnasium: A Standard Interface for Reinforcement Learning Environments* (arXiv:2407.17032). `https://doi.org/10.48550/arXiv.2407.17032`
- Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. En *Advances in Neural Information Processing Systems 32* (pp. 8026–8037).
- López González, A., & Caballero Mora, J. (2026). *Práctica Sarsa, Q-learning y Deep Q-learning* [Material del curso]. Universidad Iberoamericana.
- Anthropic. (2026). *Claude* [Modelo de lenguaje de gran escala]. `https://claude.ai`